

ODOO AI ASSISTANT

Especificación técnica y de producto del MVP

Source of Truth v1.0

20 de agosto de 2026

Estado: BASE APROBADA PARA IMPLEMENTACIÓN

Propósito del documento. Sustituye como referencia principal a los dos informes iniciales de viabilidad y arquitectura. Concreta producto, arquitectura, contratos, seguridad, instalación, milestones y estrategia de implementación para Codex. Las decisiones futuras que cambien invariantes de este documento deben registrarse mediante ADR.

Objetivo rector: construir el MVP más sencillo que demuestre correctamente el producto real sin diseñarnos un callejón sin salida.

0. Estado, alcance y control de cambios

Campo	Valor
Documento	Odoo AI Assistant - Especificación técnica y de producto del MVP
Versión	1.0
Fecha de corte	20-08-2026
Escenario de referencia	Odoo 18 Community, Linux self-hosted, PostgreSQL
Estado	Aprobado como base de implementación
Sustituye	Odoo_AI_Assistant_Viabilidad_Diseño_2026 y Odoo_AI_Arquitectura_Dinamica_Schemas_Configs_2026
Normativa de cambios	ADRs + actualización explícita de este Source of Truth cuando cambie una invariante

Cómo leer las fuentes. Las etiquetas D1/D2 indican decisiones o análisis de los PDFs originales. Las etiquetas O18/O19/CX/GH indican información comprobada en documentación oficial o repositorios al 20-08-2026. Las frases marcadas como "Decisión" son decisiones de este proyecto y no requisitos oficiales de Odoo/OpenAI.

0.1 Cambios críticos respecto a los documentos iniciales

Tema	Antes	Decisión v1.0
Persistencia	SQLite para el sidecar	PostgreSQL: DB separada del ERP, preferiblemente en el mismo cluster; sin acceso SQL del sidecar a la DB Odoo.
Sidecar	Opcional / ERP-only posible	Assistant Service obligatorio para una instalación soportada FULLY_READY; no es una segunda UI.
Logs	Capability degradable	LogProvider obligatorio para FULLY_READY; si no hay logs, la instalación queda DEGRADED/UNSUPPORTED para diagnóstico completo.
Source	Capability opcional	Source legible obligatorio para FULLY_READY en el escenario self-hosted soportado.
Capabilities	Instancia y usuario mezclados	Separar InstanceCapabilities de EffectiveRequestPolicy.
Schema cache	instance/model/revision	EffectiveModelSchema bajo usuario/policy; cache corta e identificada por contexto de seguridad.
Codex tools	MCP/App Server en abstracto	Codex App Server vía stdio; dynamicTools para el prototipo, aislado dentro de CodexAppServerEngine.
Instalación	Addon + sidecar manual	Paquete/bootstrap único que automatiza SO/DB/systemd/addon; Odoo gestiona el resto. Manual sólo como fallback.

0.2 Invariantes de proyecto

- El usuario no debe responder preguntas técnicas que el sistema puede descubrir por pantalla, ORM, configuración, source o logs.
- El LLM razona y selecciona herramientas; el software determina identidad, permisos, datos, schemas, límites, ejecución y evidencia.
- El Assistant Service nunca accede directamente por SQL a la base de datos productiva de Odoo.
- No se usa sudo() en los caminos normales de tools del agente.
- No existe shell libre, SQL libre, Python arbitrario ni execute_method genérico expuesto al modelo.
- Toda operación con efectos debe estar validada y, cuando corresponda, ligada a una aprobación del payload exacto.
- Los servicios de aplicación no contienen condicionales de mayor de Odoo salvo una excepción documentada por ADR.
- Los schemas de modelos Odoo concretos son datos runtime, no clases estáticas por versión.
- Codex es el motor inicial, no una dependencia arquitectónica del producto.
- La UI, configuración y operación cotidiana se realizan desde Odoo.

Índice

1. Visión del producto
2. Usuarios, UX y principios de comportamiento
3. Casos de uso y workflows
4. Alcance del MVP
5. Arquitectura global
6. Addon Odoo
7. Assistant Service
8. Instalación y lifecycle
9. Persistencia PostgreSQL
10. Identidad, delegación y permisos
11. Runtime schemas
12. InstanceProfile, capabilities y configuración
13. Tools y ejecución
14. ReasoningEngine y Codex App Server
15. Retrieval y RAG
16. Scanner de source
17. Logs
18. Conversaciones, contexto y memoria
19. Acciones y aprobaciones
20. Seguridad y threat model
21. Compatibilidad Odoo 18/19/futuro
22. Reutilización de repositorios y licencias
23. Estructura final del repositorio
24. Contratos internos
25. Modelo de datos
26. Flujos end-to-end
27. Estrategia de testing
28. Observabilidad
29. Roadmap y milestones
30. Criterios de aceptación

- 31. Decisiones arquitectónicas
- 32. Decisiones pospuestas
- 33. Riesgos conocidos
- 34. Estrategia y prompts para Codex
- 35. Fuentes y referencias

1. Visión del producto

Definición. Odoo AI Assistant es un agente integrado en Odoo capaz de entender el contexto real de una instalación, consultar y operar sobre Odoo, investigar código/configuración/logs/documentación y ayudar tanto a usuarios funcionales como técnicos sin exigirles conocer la arquitectura interna.

Diferencial. No se vende como "un chat dentro de Odoo". El valor está en combinar contexto automático, evidencia de la instalación concreta, capacidad agéntica acotada, diagnóstico técnico y operación segura.

1.1 Problemas que resuelve

- Preguntas funcionales: cómo ejecutar un procedimiento real en esta instalación, no en un Odoo genérico.
- Consultas de datos: responder usando ORM y agregación server-side, bajo permisos reales.
- Explicación causal: relacionar registro actual, módulos, source, XML y configuración.
- Diagnóstico: cruzar estado runtime con logs y source sin pegar megabytes al modelo.
- Acciones: crear o modificar registros y ejecutar acciones de negocio curadas, con preview/approval cuando corresponda.
- Soporte: reducir el tiempo necesario para comprender personalizaciones y errores de un cliente.

1.2 Qué NO pretende resolver en v1

- Generar o instalar módulos Odoo automáticamente.
- Administrar el sistema operativo mediante shell libre.
- Ser una plataforma multi-tenant SaaS centralizada.
- Reemplazar una herramienta de observabilidad completa.
- Prometer compatibilidad automática con cualquier versión/hosting sin adapters.
- Indexar todos los datos vivos de Odoo en una base externa.
- Convertir la arquitectura en un framework universal de agentes.

Base: D1 §§1-3,16; D2 §§1-3. Odoo 19 confirma el patrón de agente contextual y separación manager/worker (O19-AI, O19-ACT).

2. Usuarios, UX y principios de comportamiento

2.1 Usuarios objetivo

Perfil	Necesidad	Experiencia esperada
Usuario funcional	Cómo hacer X, consultar datos, pedir cambios sencillos	Lenguaje funcional. El sistema descubre modelo, vista, versión, campos y módulos sin preguntarlos.
Consultor / técnico Odoo	Explicar personalizaciones, errores, configuración y source	Puede ver evidencia técnica, archivos/líneas, logs y capability report.
Administrador	Configurar fuentes, Codex, acciones y estado del runtime	Settings progresivos: normal primero, avanzado cuando sea necesario.

2.2 Regla de autocontexto

Regla de producto. Antes de preguntar al usuario, el sistema debe intentar resolver automáticamente la información mediante ScreenContext, ORM, runtime schema, módulos, source, logs y knowledge. Sólo se pregunta cuando existe una ambigüedad funcional real o una confirmación de efecto.

USUARIO: "¿Por qué no puedo validar esta factura?"

NO:

"¿Qué versión de Odoo? ¿Qué modelo? ¿Qué vista? ¿Dónde están los logs?"

Sí:

Capturar pantalla -> leer account.move actual -> schema/permisos -> módulos -> logs recientes -> source relevante -> responder.

2.3 UX principal

- Icono en systray para abrir/cerrar un panel lateral persistente.
- Panel Owl como main component para que siga disponible mientras el usuario navega.
- App/página "AI Assistant" para historial, knowledge, diagnóstico de instalación y Settings.
- Conversaciones persistentes por usuario; cada turno usa el ScreenContext actual.
- Indicadores de evidencia: Comprobado, Inferido, Conocimiento general, No comprobado.
- Las operaciones sensibles muestran preview entendible; nunca argumentos técnicos crudos como requisito de UX.

Verificado: Odoo 18 expone registries de systray, main_components y services; la web es Owl y el action service mantiene la pila de acciones (O18-WEB). Odoo 19 AI usa contexto completo del registro (O19-CTX).

3. Casos de uso y workflows

Workflow	Objetivo	Tools típicas	Writes
HOW_TO	Explicar cómo hacer algo en la instalación actual	runtime schema, menus/views, docs, source si hace falta	No
QUERY	Consultar y agregar datos vivos	search, read, aggregate, schema	No
EXPLAIN	Explicar por qué ocurre un comportamiento	record, symbols, XML, source	No
DIAGNOSE	Investigar error/fallo real	record, logs, source, metadata	No
ACTION	Crear/modificar o ejecutar acción curada	resolve, preview, policy, approval, commit	Sí

3.1 HOW_TO

Pregunta: "¿Cómo hago una factura rectificativa de esta factura?"

El sistema intenta comprobar:

- registro/pantalla actual
- Odoo y módulos instalados
- tipo de documento, diario y campos reales
- menús/vistas disponibles
- procedimientos del cliente en knowledge
- customizaciones que cambien el flujo

Salida: pasos adaptados a la instalación + evidencias + limitaciones.

3.2 QUERY

Pregunta: "¿Cuántas facturas pendientes tiene este cliente?"

ScreenContext -> cliente actual -> ModelSchema -> AggregateSpec -> ORM server-side -> resultado pequeño -> LLM explica/presenta.

3.3 EXPLAIN / DIAGNOSE

Pregunta: "¿Por qué al confirmar este pedido se crea una tarea?"

sale.order#4832 -> action_confirm -> overrides instalados -> custom module -> source excerpt -> condición -> project.task -> respuesta con citas.

3.4 ACTION

Pregunta: "Cambia el comercial de estos presupuestos a Laura."

selected_ids -> resolver Laura -> validar user_id + schema + ACL ->
preview exacto -> aprobación -> write ORM -> relectura -> resultado.

3.5 Política de preguntas al usuario

- No preguntar por versión, modelo técnico, campo, vista, módulo, ruta de logs o base de datos si son descubribles.
- Sí preguntar por intención funcional ambigua: por ejemplo, factura independiente vs factura desde pedido.
- Sí pedir aprobación cuando una operación cambia datos y la policy lo requiere.
- Si falta evidencia técnica, el agente usa tools antes de preguntar.
- Si tras investigar sigue sin poder determinar algo, lo declara y explica qué dato falta.

4. Alcance del MVP

4.1 Incluido

- Odoo 18 Community, Linux self-hosted, como plataforma oficial inicial.
- Addon instalable con panel contextual, app/settings, setup wizard e historial.
- Assistant Service local obligatorio para instalación FULLY_READY.
- Codex App Server instalado en el mismo servidor y gestionado desde Odoo.
- Autodescubrimiento de instancia, módulos, addons_path, source y logs.
- Runtime schemas a partir de fields_get/metadata, bajo usuario real.
- Scanner de manifests, Python AST, XML y CSV de seguridad.
- PostgreSQL FTS/lexical retrieval para docs y metadatos; sin vector DB.
- Agent loop con tools read-only autónomas y límites.
- HOW_TO, QUERY, EXPLAIN y DIAGNOSE suficientemente utilizables.
- ACTION básica: create/update genérico seguro y al menos una business action curada como prueba del patrón.
- Preview + approval ligado al payload para efectos sensibles.
- Conversaciones persistentes y observabilidad lógica por trace_id.
- Instalador/bootstrap automatizado y fallback manual documentado.

4.2 Excluido explícitamente

- unlink/delete genérico por defecto.
- shell, SQL o Python arbitrario para el agente.
- generación de módulos por el agente.
- vector DB, embeddings obligatorios, knowledge graph y reranker.
- Redis/Kafka/colas distribuidas/Kubernetes.
- multi-tenant SaaS y multi-instancia centralizada.
- roles/políticas empresariales sofisticadas, DLP/KMS, SSO/SCIM.
- soporte completo Odoo.sh/Odoo Online.
- Odoo 19 como requisito de salida del primer slice; se añade después con la misma contract suite.

5. Arquitectura global

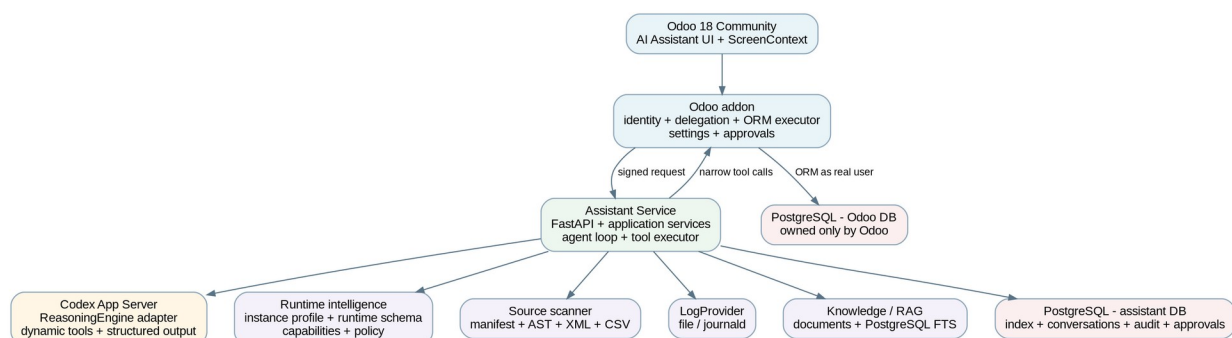


Figura 1. Arquitectura de referencia del MVP.

5.1 Boundaries

Boundary	Responsabilidad	No debe hacer
Browser/Owl	Captura navegación y UX	No decidir permisos ni confiar en user_id enviado por JS.
Addon Odoo	Identidad, ORM, settings, approvals, delegación	No ejecutar LLM ni scanner pesado.
Assistant Service	Orquestación, retrieval, tools, storage, observabilidad	No SQL directo a DB Odoo; no sudo.
ReasoningEngine	Razonar y seleccionar tools	No conocer detalles de transporte Odoo ni tener autoridad propia.
Source/Log adapters	Obtener evidencia del host	No recibir instrucciones libres del LLM.
Assistant PostgreSQL	Persistencia propia	No convertirse en réplica de datos vivos Odoo.

5.2 Dependencias permitidas

```

UI -> Odoo addon controllers/services
Odoo addon -> internal HTTP contract to Assistant Service
Application -> contracts + ports
Adapters -> contracts + ports + external technologies
ReasoningEngine adapter -> ContextPack + ToolSpec + AnswerEnvelope
ToolExecutor -> policy + adapters
Storage -> repositories, never application-specific UI

```

```

PROHIBIDO:
application -> Odoo version client
engine -> filesystem/log implementation
browser -> Assistant Service directly (MVP)
service -> Odoo production SQL

```

6. Addon Odoo

6.1 Responsabilidades

- Registrar systray, main component/panel y servicio JS de ScreenContext.
- Derivar identidad del usuario autenticado en servidor; nunca confiar en identidad procedente del browser.
- Exponer endpoints públicos del addon para chat/settings y endpoints internos estrechos para tools.
- Ejecutar ORM con el uid/contexto delegados sin sudo().
- Resolver ACL/record rules y los errores de acceso naturales de Odoo.
- Mantener Settings visibles y workflow de setup/health.
- Mostrar previews/aprobaciones y resultados de acciones.
- Gestionar grupos mínimos del módulo y acceso a settings.

6.2 ScreenContext

```

ScreenContext {
  action_id: int | null
  menu_id: int | null
  view_type: str | null
  model: str | null
  res_id: int | null
  selected_ids: list[int]
  allowed_context_subset: dict
  captured_at: datetime
}

# user_id, companies and security context are added server-side.

```


Regla. ScreenContext es una pista de navegación, no autoridad. Antes de usar un registro para responder o actuar, el addon lo vuelve a leer mediante ORM bajo el usuario real.

6.3 UI propuesta

Superficie	Contenido
Panel lateral	chat, streaming/polling de eventos, fuentes, previews, acciones
AI Assistant > Conversations	historial personal, búsqueda, estado
AI Assistant > Knowledge	documentos/fuentes indexadas
AI Assistant > Diagnostics	estado de runtime, logs/source, scan, últimas trazas fallidas
Settings > AI Assistant	estado general, Codex, capabilities, acciones, avanzado

6.4 Comunicación UI -> service

Para el MVP se evita exponer el Assistant Service al navegador. El browser habla sólo con Odoo. Los turns largos se implementan como trabajo asíncrono + polling de eventos:

```
Browser POST /odoo_ai/turn
-> addon captura identidad/contexto
-> POST http://127.0.0.1:<service>/v1/turns
<- turn_id

Browser GET /odoo_ai/turn/<id>/events?after=<seq>
-> addon proxy
-> service event stream persisted/buffered

# SSE puede añadirse después sin cambiar el contrato del turn.
```

7. Assistant Service

7.1 Tecnología

Área	Elección MVP	Motivo
Lenguaje	Python 3.12+	Ecosistema, Codex tooling, parsers y velocidad de desarrollo.
HTTP	FastAPI + Uvicorn	Contratos Pydantic, async y endpoints internos simples.
DB	PostgreSQL + SQLAlchemy 2 + Alembic	Mismo motor operacional que Odoo, DB aislada y migraciones estándar.
Reasoning	Codex App Server	Máximo leverage del prototipo; login ChatGPT y agent loop disponibles.
Retrieval	Postgres FTS + índice estructural	Sin dependencia vectorial; explicable y suficiente para MVP.

7.2 Responsabilidades

- Bootstrap de InstanceProfile y capabilities.
- Persistencia de conversaciones, índices, scans, approvals, audit y trazas.
- Construcción de ContextPack y retrieval.

- Gestión del agent loop y Codex App Server.
- Registro simple de tools y ToolExecutor.
- Scanner de source y providers de logs/docs.
- Redacción de secretos y normalización de evidencia.
- Exposición de health/setup APIs consumidas exclusivamente por el addon en MVP.

7.3 Estado de readiness

Estado	Significado
FULLY_READY	Service, assistant DB, Codex, source y LogProvider operativos; scan válido.
DEGRADED	Puede responder parcialmente, pero falta una capability obligatoria del perfil soportado.
ERROR	No se puede iniciar turns de forma segura/confiable.

8. Instalación y lifecycle

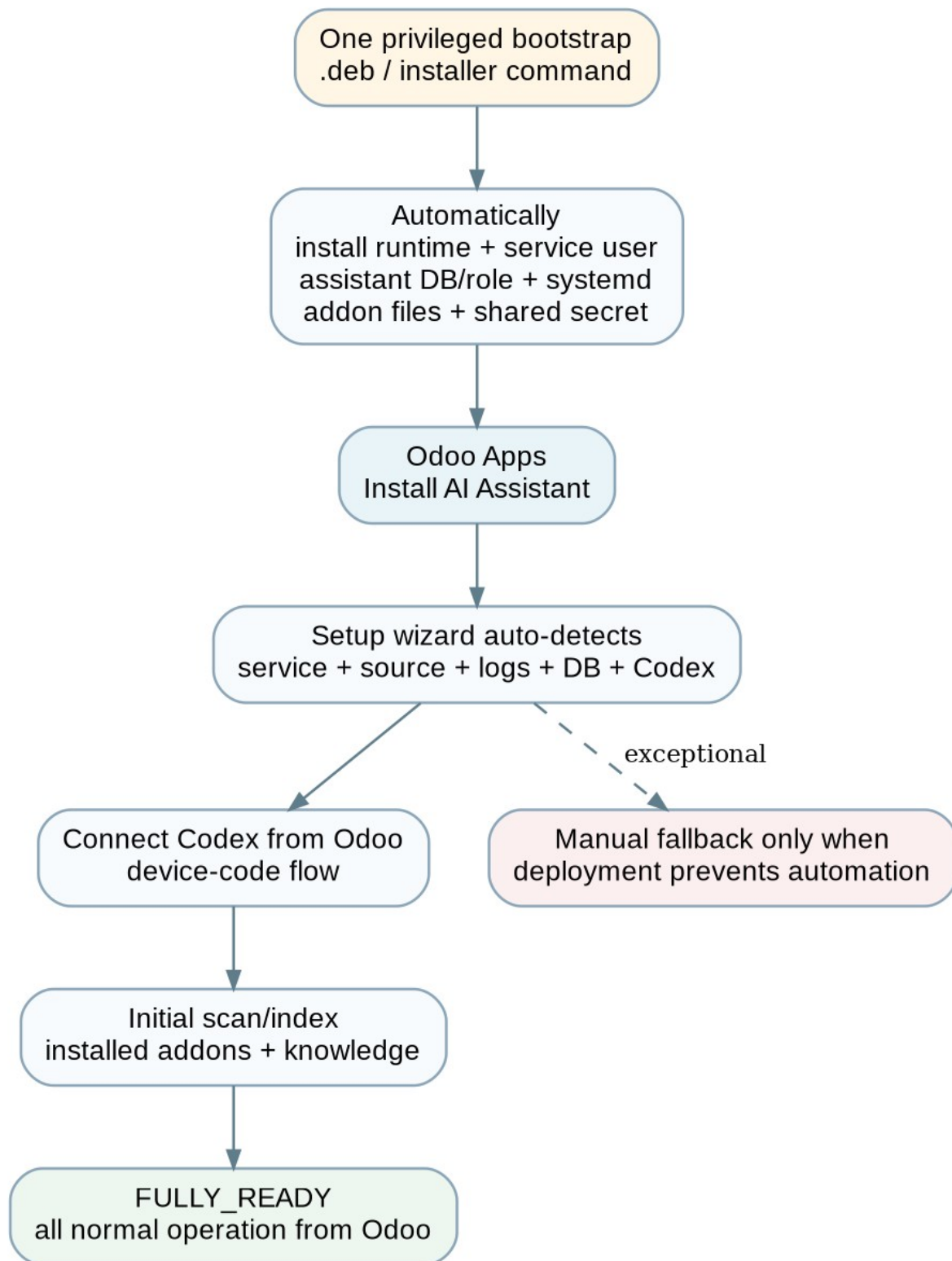


Figura 2. Instalación orientada a producto: un paso privilegiado, operación posterior desde Odoo.

8.1 Principio

Decisión. No se concede root al proceso Odoo. Una activación de módulo no puede crear de forma segura usuarios del SO, servicios systemd y roles PostgreSQL. Por tanto, el producto automatiza todo lo posible mediante un único bootstrap privilegiado previo o paquete del sistema; el cliente no configura manualmente cada pieza.

8.2 Camino recomendado

1. Instalar el paquete/bootstrap del runtime una sola vez (ideal: paquete .deb; fallback: instalador firmado).
2. El instalador detecta o recibe la ruta de odoo.conf y localiza addons_path, servicio Odoo y PostgreSQL.
3. Crea usuario/directorios del runtime, DB/role del Assistant, unit systemd, secretos compartidos y coloca el addon.
4. Reinicia/recarga Odoo si es necesario y deja el servicio local escuchando sólo en loopback.
5. En Odoo > Apps se instala AI Assistant como cualquier módulo.
6. El setup wizard verifica automáticamente service/DB/source/logs y muestra únicamente excepciones.
7. El administrador conecta Codex desde Settings mediante device-code flow.
8. Se ejecuta el scan inicial y la instalación pasa a FULLY_READY.

8.3 Fallback manual

Si un hosting impide automatizar alguno de los pasos, el instalador debe generar una guía exacta para esa instancia. El fallback puede requerir crear el servicio, conceder lectura a logs/source o configurar la DB, pero no es el flujo normal de producto.

8.4 Uninstall / upgrade

- Actualizar addon y runtime de forma coordinada mediante versionado de contrato interno.
- Migraciones assistant DB forward-only con backup previo.
- Desinstalar el addon no borra automáticamente la assistant DB; un comando explícito puede purgarla.
- El runtime puede actualizarse sin reiniciar Odoo salvo que cambie el contrato del addon.
- El scan se invalida por fingerprints, no se destruye la DB en cada actualización.

9. Persistencia PostgreSQL

9.1 Topología elegida

```
PostgreSQL cluster (default self-hosted)
├── <odoo_database>      owner: odoo
└── odoo_ai              owner: odoo_ai_service
```

```
odoo_ai_service role:
  CONNECT odoo_ai: YES
  CONNECT <odoo_database>: NO
```

Odoo live data always travels through the addon/ORM boundary.

9.2 Por qué no SQLite

Criterio	SQLite	DB PostgreSQL separada
Operación	Segundo motor, backups/lifecycle separados	Reutiliza cluster/herramientas ya presentes
Aislamiento	Bueno	Bueno: DB/role independientes
Concurrencia	Limitada	Suficiente para conversaciones/scans futuros
FTS	FTS5 fuerte	FTS nativo + pg_trgm opcional
Evolución	Probable migración futura	Permite pgvector opcional sin cambiar de motor
Hosting alternativo	Muy portable	Puede apuntar a PostgreSQL externo sin cambiar código

9.3 Qué se persiste y qué no

Persistir	No persistir como copia
conversaciones, summaries, traces, approvals, audit	saldos/estados de negocio como cache permanente
source index, fingerprints, symbols, XML metadata	todos los registros de Odoo
doc chunks + FTS	logs completos indefinidamente
capability snapshots, scan runs	secretos Codex/Odoo en prompts o tablas de chat

10. Identidad, delegación y permisos

10.1 Principio

Invariante de seguridad. El agente no es una cuenta técnica superuser. Toda lectura/escritura Odoo se ejecuta con la identidad efectiva del usuario que inició el turn y se vuelve a validar en cada tool call.

10.2 DelegationContext

```
DelegationContext {
  request_id: UUID
  user_id: int
  company_id: int
  allowed_company_ids: list[int]
  lang: str | null
  scopes: list[str]
  issued_at: datetime
  expires_at: datetime
  nonce: str
}

# Serialized and signed by the addon with an install-time shared secret.
# The service cannot mint a valid delegation for another user.
```

10.3 Ejecución de una tool Odoo

9. ToolExecutor valida ToolSpec, schema, request policy y budgets.
10. El service llama un endpoint interno estrecho del addon con DelegationContext firmado.
11. El addon valida firma, expiración, nonce/scopes y reconstruye el Environment con uid/contexto.
12. El executor local sólo permite operaciones conocidas; no recibe un método arbitrario.
13. Odoo aplica ACL, record rules, field restrictions y reglas de negocio del código ejecutado.
14. El resultado se normaliza y vuelve como RecordSnapshot/Evidence.

10.4 InstanceCapabilities vs EffectiveRequestPolicy

```
InstanceCapabilities:
  runtime_orm: true
  runtime_schema: true
  source_read: true
  logs_file: true
  logs_journal: false
  codex_engine: true

EffectiveRequestPolicy (por turn):
  user_id: 17
  companies: [1]
  allowed_tools: [...]
  writes_enabled: false|true
  max_rows: 100
```

```
max_tool_calls: 12
technical_evidence_visible: true|false
```

11. Runtime schemas

11.1 Decisión

No se crean clases SaleOrder18/SaleOrder19/AccountMove18. El schema útil se obtiene de la instalación real. Odoo 18 documenta fields_get() y metadata en ir.model.fields; además, los fields restringidos por grupo desaparecen de fields_get() para usuarios sin acceso.

Fuentes verificadas: O18-ORM, O18-API, O18-SEC. D2 §§3,6.

11.2 Dos niveles de schema

Tipo	Uso	Seguridad
InstanceModelCatalog	Index técnico y descubrimiento de modelos/módulos	No se expone directamente al LLM como autoridad de campos accesibles.
EffectiveModelSchema	Tool schemas y validación del turn actual	Se obtiene/filtra bajo usuario, company context y policy.

11.3 ModelSchema / FieldSpec

```
FieldSpec {
  name: str
  label: str
  type: str
  relation: str | null
  required: bool
  readonly: bool
  selection: list[tuple[str, str]] | null
  help: str | null
}

ModelSchema {
  schema_id: str
  model: str
  revision: str
  fields: dict[str, FieldSpec]
  source: "runtime"
  captured_for_user: int
  policy_revision: str
}
```

11.4 Cache e invalidación

- MVP: cache in-memory corta (por ejemplo 60-300 s) para EffectiveModelSchema.
- Key incluye instance, model, user_id, allowed_company_ids hash, policy_revision e installed_modules_hash.
- Persistir fingerprints estructurales, no el schema efectivo de cada usuario indefinidamente.
- Botón Reanalizar sistema fuerza scan/fingerprint refresh.
- Cambio de módulos o metadata relevante invalida schemas derivados y tools que dependan de ellos.

12. InstanceProfile, capabilities y configuración

12.1 InstanceProfile

```
InstanceProfile {
  instance_id: UUID
  odoo_version: "18.0"
  major: 18
  edition: "community"
  deployment: "linux_systemd"
```

```

database_name: "... " # metadata, not credential
addons_paths: [...]
installed_modules_hash: "... "
log_provider: "file" | "journald"
profile_revision: "... "
}

```

12.2 Descubrimiento

15. Addon runtime: versión, DB, módulos instalados, user/company context.
16. Configuración Odoo: addons_path, data_dir y log destination sin exponer secretos.
17. Filesystem: verificar lectura únicamente sobre roots derivados/configurados.
18. Logs: detectar file provider; fallback journald cuando exista y tenga permisos.
19. PostgreSQL Assistant: health/migration revision.
20. Codex: binary, app-server, account state y rate limits.
21. Generar InstanceCapabilities con estado DETECTED/NO_PERMISSION/NOT_FOUND/ERROR.

12.3 Settings progresivos

Nivel normal	Advanced
Estado: FULLY_READY/DEGRADED	service endpoint, DB revision
Codex: conectado / conectar	CODEX_HOME, binary/version, engine diagnostics
Source: disponible / reanalizar	source roots, include/exclude
Logs: disponible	provider, unit/file path, windows/limits
Knowledge: fuentes + index	chunk/FTS diagnostics
Capabilities: preguntas/queries/diagnose/actions	budgets, policy revision, trace retention

13. Tools y ejecución

13.1 Principios

- Tools pequeñas, con schemas explícitos y resultados estructurados.
- Read-only tools pueden ejecutarse autónomamente durante el agent loop.
- Cada tool vuelve a validar usuario/policy; el schema expuesto al modelo no es autorización.
- Los límites se aplican server-side, no como sugerencias en prompt.
- No existe execute_kw/execute_method genérico expuesto al engine.

13.2 Catálogo MVP

Tool	Uso	Riesgo
odoo.current_record	Releer registro(s) de pantalla con campos curados	read
odoo.read_records	Leer IDs/fields validados	read
odoo.search_records	DomainSpec validado + limit	read
odoo.aggregate	count/sum/avg/groupby server-side	read
odoo.get_model_schema	EffectiveModelSchema	metadata
odoo.find_menu_paths	Rutas de menú accesibles para HOW_TO	metadata
odoo.inspect_view	Metadata/arquitectura de vista relevante	metadata
odoo.installed_modules	Módulos instalados/fingerprint	metadata

Tool	Uso	Riesgo
source.find_symbol	Modelo/método/XML id -> candidatos	read
source.find_model_extensions	_inherit/_name y módulos relacionados	read
source.read_excerpt	Fragmento acotado por ref	read
logs.search	Ventana + términos + caps	read
logs.read_traceback	Traceback por fingerprint/ref	read
docs.search	FTS sobre knowledge	read
write.preview_create	Propuesta create validada	write-preview
write.preview_update	Propuesta update validada	write-preview
write.commit	Ejecuta Approval ligado al payload	write
business.preview_action	Acción curada allowlisted	action-preview
business.commit_action	Ejecuta acción aprobada	action

13.3 Structured query

```
DomainSpec {
  conditions: [
    {field: "partner_id", op: "=", value: 42},
    {field: "payment_state", op: "in", value: ["not_paid", "partial"]}
  ]
  limit: 50
  order: [{field: "invoice_date", direction: "desc"}]
}

# Fields/operators validated against EffectiveModelSchema.
# ORM record rules remain authoritative.
```

13.4 Tool registry simplificado

El MVP no implementa un plugin framework. ToolCatalog puede ser un registro explícito en Python y factories que filtran según workflow/capabilities/policy. La abstracción se amplía cuando existan extensiones reales.

14. ReasoningEngine y Codex App Server

14.1 Port estable

```
class ReasoningEngine(Protocol):
    async def run_turn(
        self,
        context: ContextPack,
        tools: list[ToolSpec],
        output_schema: dict,
    ) -> AnswerEnvelope: ...
```

14.2 CodexAppServerEngine

- El Assistant Service inicia/controla codex app-server por stdio (JSONL), transporte principal documentado.
- El login se inicia desde Odoo Settings mediante account/login/start type=chatgptDeviceCode; Odoo muestra verificationUrl y userCode.
- Cada turn del producto crea una sesión/thread Codex desechable o aislada; la memoria real permanece en nuestra assistant DB.
- Se envía ContextPack compacto y un outputSchema para AnswerEnvelope.
- Para el prototipo se usan dynamicTools de App Server. Son experimentales, por lo que todo el acoplamiento queda confinado al adapter.

- Si dynamicTools dejan de ser adecuadas, CodexAppServerEngine puede migrar a MCP sin tocar application/retrieval/Odoo.

Verificado: CX-APP documenta stdio, account/login/start, device-code, outputSchema y dynamicTools/item/tool/call. DynamicTools está marcado experimental al 20-08-2026.

14.3 Loop de un turn

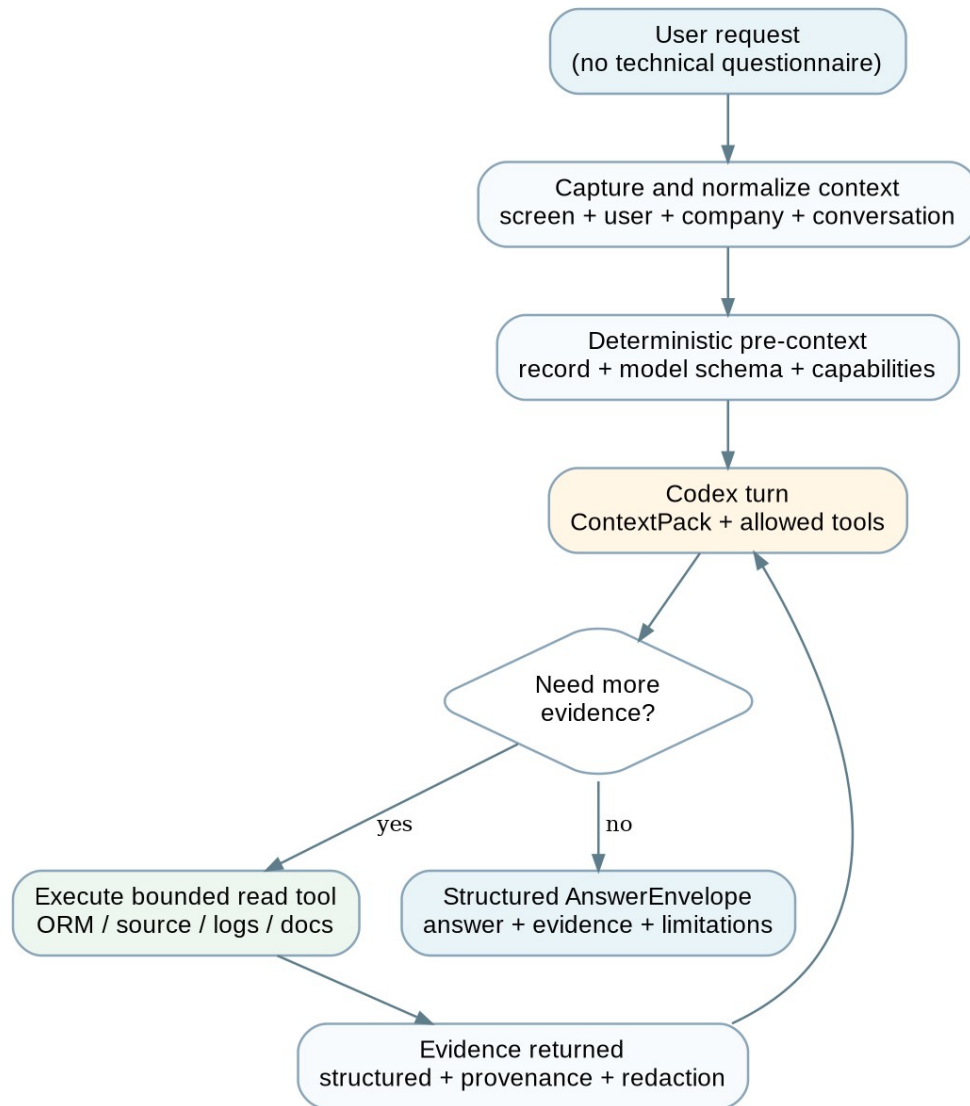


Figura 3. Turn agéntico: contexto inicial pequeño, tools bajo demanda y evidencia estructurada.

14.4 Sandbox de Codex

Requisito. Codex no debe leer source/logs por shell saltándose ToolExecutor. En Linux se configura un permission profile restrictivo y un workspace vacío; network de comandos deshabilitado; approvalPolicy=never para comandos. El instalador ejecuta un self-test que intenta leer rutas prohibidas y falla la instalación si el aislamiento no se aplica.

Durante el prototipo interno puede aceptarse una política temporal menos fuerte, pero antes de entregar a clientes el isolation test debe ser obligatorio. El product threat model asume que el único acceso a source/logs del modelo son nuestras tools redactoras y acotadas.

14.5 Auth y producto comercial

El login ChatGPT/Codex es adecuado para el prototipo y pilotos operados por el titular autorizado de la cuenta. No se diseña la arquitectura comercial alrededor de credenciales personales. ReasoningEngine permite sustituir Codex por una oferta API/business sin reescribir el núcleo. Esta observación es técnica/operativa, no asesoramiento jurídico.

15. Retrieval y RAG

15.1 Estrategia

Decisión. Retrieval desde el primer día; vector DB no. Para source se priorizan símbolos/relaciones; para docs se usa FTS lexical. Embeddings se añaden sólo si evals reales muestran pérdida de recall semántico.

15.2 Qué indexar vs leer vivo

Indexar/cachar	Consultar bajo demanda
manifest, hashes, módulos, símbolos, XML ids, relaciones model/view	estado de registros, saldos, permisos efectivos
metadata estructural relativamente estable	fields_get efectivo si una tool depende de ello
doc chunks, títulos, secciones	logs recientes
resúmenes técnicos ligados a fingerprint	resultados de acciones y verificaciones

15.3 Motores MVP

Corpus	Baseline
Código Odoo	índice estructural: model/method/module/XML id + exact/trigram
Documentación	PostgreSQL tsvector + ts_rank_cd
Metadata Odoo	tablas/índices exactos
Logs	provider search temporal + keyword/traceback parser; no ingest completo

15.4 ContextPack

```
ContextPack {
  request: UserRequest
  screen: ScreenContext
  user: UserExecutionContext
  workflow_hint: Workflow | null
  instance: InstanceProfileSummary
  live_evidence: list[Evidence]
  retrieved_evidence: list[Evidence]
  conversation_state: ConversationState
  limits: TurnLimits
}
```

Initial target: 3-8 useful evidence items, not the entire repository/log.

16. Scanner de source

16.1 Scope

- Sólo roots configurados/detectados de addons; nunca escaneo recursivo del host.
- Priorizar módulos instalados; módulos disponibles no instalados pueden indexarse bajo demanda.
- No importar/ejecutar código del addon durante el scan estático.
- Hashes/mtime evitan reprocesar archivos sin cambios.

16.2 Extractores

Fuente	Extraer	Limitación
__manifest__.py	name, version, depends, data, assets, license	usar ast.literal_eval; manifests dinámicos se marcan no evaluables
Python AST	_name/_inherit, fields, methods, decorators, imports, file/lines	no resuelve metaprogramación/monkey patches runtime
XML	records, views, inherit_id, xpath, actions, menus, groups	estado final depende de herencias/orden
CSV security	ACL declaradas	runtime puede diferir por otros módulos

16.3 Source IR

```
SourceSymbol {
  module: "custom_sale"
  kind: "method"
  model: "sale.order"
  name: "action_confirm"
  path: "custom_sale/models/sale_order.py"
  start_line: 42
  end_line: 71
  fingerprint: "sha256:..."
}
```

16.4 Provenance

La respuesta no debe decir "custom" sólo porque el directorio se llame custom. Clasificación recomendada: checkout oficial -> repo OCA conocido -> remote/manifest conocido -> regla manual -> third_party_or_custom.

17. Logs

17.1 Requisito de producto

Decisión. Una instalación soportada debe tener LogProvider funcional. Si no se pueden leer logs, el setup no marca FULLY_READY. Esto no significa copiar todos los logs a la assistant DB.

17.2 Providers

Provider	MVP	Descubrimiento
FileLogProvider	Sí, preferido	logfile/config/path detectado
JournalLogProvider	Sí cuando Odoo usa systemd	unit detectada + permisos concedidos por installer
DockerLogProvider	Posterior	deployment profile
OdooShLogProvider	Posterior	adapter específico de plataforma

17.3 API de búsqueda

```
LogSearchRequest {
  from_ts: datetime | null
  to_ts: datetime | null
  terms: list[str]
  max_lines: int <= 200
}
```

```

    max_bytes: int <= configured_cap
}

LogEvidence {
    provider: str
    timestamp_range: ...
    excerpt: str
    traceback_fingerprint: str | null
    correlation: "direct" | "temporal_inference"
}

```

17.4 Reglas

- Buscar alrededor de la acción/turno y términos relevantes antes que leer megabytes.
- Extraer traceback completo con contexto acotado.
- Agrupar repeticiones por fingerprint.
- Si la correlación con el registro es temporal/heurística, etiquetarla como inferencia.
- Redactar secretos/tokens/credenciales antes de enviar evidencia al engine.

18. Conversaciones, contexto y memoria

18.1 Fuente de verdad

La memoria de producto vive en la assistant DB, no en el thread de Codex. Esto permite cambiar de engine, reiniciar Codex y controlar retención.

18.2 ConversationState

```

ConversationState {
    current_screen: ScreenContext | null
    mentioned_records: list[RecordRef]
    pending_approval_id: UUID | null
    short_summary: str
    last_user_intent: str | null
}

```

18.3 Contexto por turn

- Últimos mensajes necesarios + summary incremental.
- ScreenContext actual capturado al enviar el mensaje.
- Referencias estructuradas a entidades mencionadas.
- No reenviar 200 mensajes ni convertir el chat en la única memoria técnica.
- Hallazgos reusables se guardan como evidence/index ligados a fingerprints, no como "recuerdo" narrativo incuestionable.

19. Acciones y aprobaciones

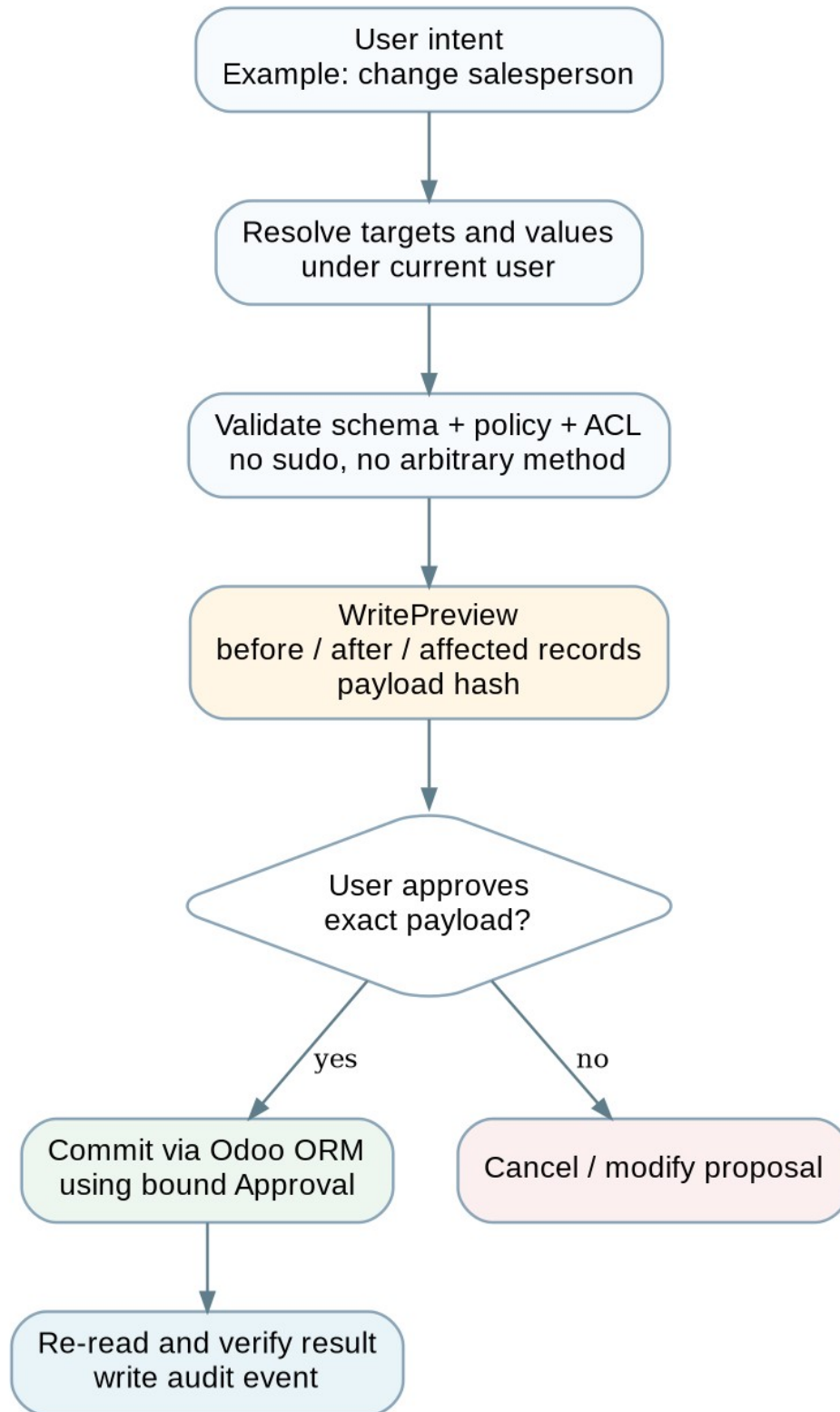


Figura 4. Patrón de efectos: *proposal -> preview -> approval -> commit -> verify*.

19.1 MVP de escritura

- Generic create/update para modelos/campos permitidos por policy y EffectiveModelSchema.
- Delete/unlink deshabilitado por defecto.
- Business actions sólo mediante handlers explícitos/allowlist; no model.method arbitrario.
- Preview muestra targets, valores actuales, valores propuestos y consecuencias conocidas.
- Approval token firmado/hash ligado al payload canónico y con expiración/consumo único.
- Commit relea el registro y devuelve verificación.

19.2 Contratos

```
WriteRequest {
  operation: "create" | "update"
  model: str
  ids: list[int]
  values: dict
}

WritePreview {
  preview_id: UUID
  request: WriteRequest
  before: list[RecordSnapshot]
  normalized_values: dict
  payload_hash: str
  risk: "low" | "medium" | "high"
  expires_at: datetime
}

Approval {
  approval_id: UUID
  preview_id: UUID
  payload_hash: str
  user_id: int
  approved_at: datetime
  expires_at: datetime
  consumed_at: datetime | null
}
```

19.3 Reutilización ERPipe

ERPipe/mcp-odoo ya implementa un flujo preview_write -> validate_write -> execute_approved_write con fields_get live, token same-session, expiración/consumo y confirmación. Al ser MIT, es el candidato principal para adaptar ideas y partes de implementación de write safety, sin adoptar su arquitectura completa de conexión Odoo externa.

20. Seguridad y threat model

20.1 Activos y fronteras

Riesgo	Mitigación MVP
LLM se convierte en superusuario	delegación de uid real + no sudo + tools estrechas
SQL destructivo/leak	sin acceso SQL a Odoo y sin SQL tool
Método público inseguro	sin execute_method genérico; business actions allowlisted
Prompt injection en source/log/docs	evidence tratada como datos no confiables; tool authority separada
Secretos en prompts	Redactor + no leer config sensible completa + regex/known-key filtering
Codex usa shell para saltarse tools	permission profile restrictivo + empty workspace + self-test de aislamiento
Replay de delegación	exp corto + nonce + request id + firma
Approval desacoplada	payload_hash + user + expiración + one-time consume
Exceso de datos	row/source/log/tool budgets server-side
Fuga multi-company	allowed_company_ids y ORM bajo usuario real

20.2 Prompt injection

- Todo texto recuperado se etiqueta como UNTRUSTED_EVIDENCE.
- Comentarios de código, logs y documentos nunca modifican políticas ni tool permissions.
- ToolSpec se construye fuera del modelo.
- El modelo puede solicitar una tool; ToolExecutor decide si se ejecuta.
- Las respuestas de tools son JSON/records normalizados y redactoras antes de volver al engine.

20.3 Seguridad pospuesta

SSO/SCIM, policy engine sofisticado, DLP/KMS, auditoría central, approvals multinivel y data residency se consideran producto maduro, no requisito del primer MVP. No se posponen las invariantes de identidad real, no-sudo, no-shell libre, no-SQL libre y payload-bound approvals.

21. Compatibilidad Odoo 18/19/futuro

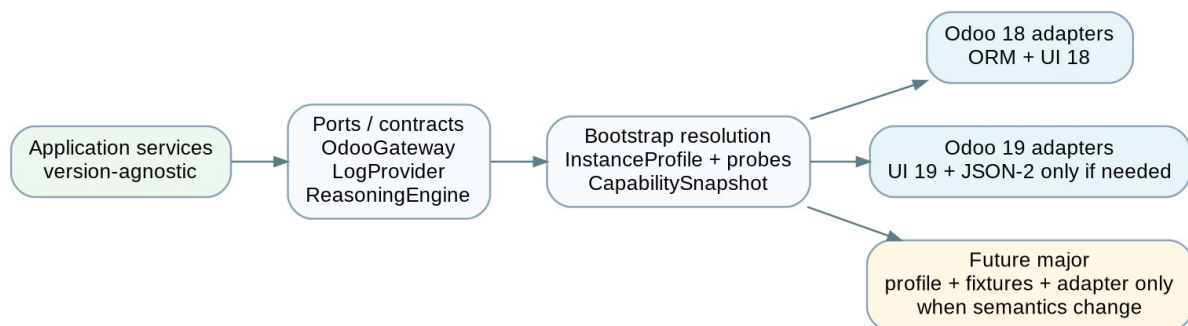


Figura 5. La versión selecciona adapters/capabilities; no entra en los servicios de aplicación.

21.1 Odoo 18

- Ruta principal del producto: addon + ORM local/delegado, no XML-RPC.
- UI Owl 18 específica en assets del addon.
- External XML-RPC sólo sería útil para integraciones externas futuras, no para el core del MVP.

21.2 Odoo 19

- Mantener el mismo contrato addon <-> service siempre que sea posible.
- Adaptar frontend si cambian APIs Owl/assets.
- JSON-2 es el reemplazo oficial de XML-RPC/JSON-RPC externos; sólo implementar adapter externo si un deployment lo necesita.
- Reutilizar patrones UX del AI oficial: contexto del registro, agentes/topics/tools/sources y separación manager/worker.

O19-RPC: XML-RPC/JSON-RPC externos programados para retirada en Odoo 22; JSON-2 es el reemplazo documentado.

21.3 Criterio para una mayor futura

Añadir Odoo N:

1. fixture /web/version + runtime probes
2. ejecutar contract suite
3. actualizar VersionProfile sólo si aporta datos reales
4. adaptar UI si es necesario
5. crear adapter sólo si cambió protocolo/semántica

ÉXITO: application/ y workflows/ no cambian.

22. Reutilización de repositorios y licencias

22.1 Estrategia

Decisión. No se adopta un repo entero como base por defecto. Se construye el producto como monorepo propio y se reutilizan selectivamente componentes/patrones de proyectos existentes cuando licencia, calidad y

acoplamiento lo permiten.

Proyecto	Licencia/estado	Reutilizar	No adoptar
erpipe-org/mcp-odoo	MIT, activo	safe writes, fields smart selection, domains/aggregate, scanner, BM25 ideas, smoke tests, diagnostics	arquitectura credentials->External API como boundary principal
Vauxoo/mcp.odoo	MIT	detección/normalización de transports 8-19+, JSON-2 ideas	tools CRUD/execute_kw generales como superficie del agente
apexive/odoo-llm	LGPL-3	ideas/UI threading/provider/tool framework; evaluar módulos aislados	framework completo, providers/vector stores no necesarios
OCA/ai	repo AGPL; módulos pueden variar	patrones OCA y piezas sólo tras revisar manifest/licencia	copiar core AGPL a producto propietario
OdooClaw	MIT	ideas de addon->gateway, session memory, retrieval/tool selection	shell/memory/model stack completo; producto distinto
Odoo 19 AI oficial	propietario Odoo	patrones UX/producto: contexto, tools/sources, manager-worker	copiar código Enterprise o depender de él en Community

22.2 Base recomendada

El repositorio base será propio. ERPipe es el mayor donor de código MIT y debe auditarse primero a nivel de archivos para extraer/adaptar funciones de scanner, query helpers y write safety. Vauxoo complementa transporte/compatibilidad. Apexive/OdooClaw aportan patrones Odoo-native. Odoo 19 AI se usa como referencia funcional/UX.

22.3 Política de licencia

- Objetivo: preservar capacidad de distribución comercial/propietaria.
- MIT/BSD/Apache: candidatos preferentes; conservar notices/atribución cuando corresponda.
- LGPL: revisar integración y obligaciones antes de copiar código; preferir ideas o módulos claramente separables.
- AGPL: no integrar código en el núcleo propietario sin revisión jurídica explícita.
- Licencias se revisan por fichero/módulo antes de copiar, no sólo por el README del repo.

23. Estructura final del repositorio

```

odoo-ai-assistant/
├── addons/
│   └── odoo_ai_assistant/
│       ├── __manifest__.py
│       ├── models/
│       │   └── res_config_settings.py
│       ├── controllers/
│       │   ├── assistant.py
│       │   └── internal_gateway.py
│       ├── services/
│       │   ├── screen_context.py
│       │   ├── delegation.py
│       │   └── orm_executor.py
│       ├── security/
│       ├── static/src/
│       │   ├── services/screen_context_service.js
│       │   └── components/assistant_panel/
│       └── views/
├── service/
│   ├── pyproject.toml
│   └── src/odoo_ai/
│       ├── api/
│       ├── contracts/
│       ├── application/
│       ├── engine/
│       └── base.py

```



```
├── codex_app_server.py
├── tools/
├── odoo/
├── source/
├── logs/
├── retrieval/
├── knowledge/
├── runtime/
├── security/
├── storage/
├── observability/
├── migrations/
├── installer/
│   ├── bootstrap/
│   ├── systemd/
│   └── debian/
├── tests/
│   ├── unit/
│   ├── contracts/
│   ├── integration/
│   ├── e2e/
│   └── fixtures/
│       └── odoo18/
├── docs/
│   ├── ARCHITECTURE.md
│   ├── adr/
│   ├── contracts/
│   └── compatibility.md
```

23.1 Reglas de dependencia

Zona	Puede depender de	No puede depender de
contracts	Pydantic/std lib	Odoo, FastAPI, Codex, storage
application	contracts + ports	version adapters, filesystem details
engine	contracts + Codex client	Odoo ORM/source/log concrete adapters
tools	contracts + policy + ports	UI
source/logs/odoo adapters	contracts + external libs	application workflow logic
addon	Odoo APIs + shared HTTP schemas	service Python package internals

24. Contratos internos

Los contratos son el lenguaje estable del producto. El source of truth ejecutable debe vivir en service/src/odoo_ai/contracts y exportar JSON Schema para tests/documentación. No se congela el nombre exacto de todas las clases, sí las responsabilidades.

Contrato	Responsabilidad
RecordRef	model, id, display_name opcional. Identidad mínima de un registro.
RecordSnapshot	RecordRef + fields normalizados + captured_at + provenance.
ScreenContext	navegación capturada por UI; sin identidad confiada del browser.
UserExecutionContext	uid/company/lang efectivo derivado por servidor.
InstanceProfile	hechos detectados de la instancia/deployment.
InstanceCapabilities	capacidades del deployment, no permisos de usuario.
EffectiveRequestPolicy	tools/budgets/writes/visibilidad aplicables al turn.

Contrato	Responsabilidad
FieldSpec / ModelSchema	metadata runtime de fields/model.
Evidence	unidad de evidencia con kind/status/provenance/pointer.
ContextPack	entrada normalizada al ReasoningEngine.
ToolSpec	nombre, description, input_schema, risk, executor id.
ToolCall / ToolResult	llamada validada y resultado estructurado.
WriteRequest / WritePreview / Approval	flujo de efectos ligado al payload.
AnswerEnvelope	respuesta final estructurada y citable.
TraceEvent	evento lógico de observabilidad.

24.1 Evidence

```
Evidence {
  evidence_id: UUID
  kind: "record" | "metadata" | "source" | "log" | "document" | "general"
  status: "checked" | "inferred" | "general" | "unknown"
  title: str
  summary: str
  payload: dict
  pointer: dict | null
  observed_at: datetime | null
  sensitivity: "normal" | "technical" | "sensitive"
  fingerprint: str | null
}
```

24.2 AnswerEnvelope

```
AnswerEnvelope {
  answer_markdown: str
  workflow: "HOW_TO" | "QUERY" | "EXPLAIN" | "DIAGNOSE" | "ACTION"
  confidence: "high" | "medium" | "low"
  evidence_refs: list[UUID]
  limitations: list[str]
  proposed_action: WritePreview | BusinessActionPreview | null
}

# The UI renders evidence refs as chips/citations.
# The model may not invent evidence ids: output is validated.
```

25. Modelo de datos

25.1 Assistant DB - entidades MVP

Tabla	Propósito	Índices
instance_profile	perfil/fingerprint actual	instance_id unique
capability_snapshot	historial de probes/readiness	instance_id, created_at
scan_run	estado de scans	status, started_at
source_file	path/hash/module	module, sha256, path
source_symbol	model/method/xml refs	model+name, module, path
xml_record	xml id/model/inherit/view metadata	xml_id, model

Tabla	Propósito	Índices
knowledge_source	documento y versión	status, fingerprint
knowledge_chunk	texto/metadata/tsvector	source_id, FTS GIN
conversation	owner user_id + title/state	user_id, updated_at
message	turn messages	conversation_id, created_at
turn	workflow/status/trace	conversation_id, status
evidence_used	evidence persistida usada en respuesta	turn_id, kind
approval	payload hash + user + expiry	payload_hash, user_id, consumed_at
audit_event	acciones y eventos sensibles	trace_id, event_type, created_at
trace_event	pipeline observability	trace_id, seq

25.2 Odoo DB

El add-on añade únicamente modelos/configuración que pertenezcan a la experiencia Odoo (settings, grupos y, si conviene, metadata ligera de instalación). No se almacenan índices del scanner ni copias de logs en tablas Odoo. El Assistant Service no recibe credenciales SQL de la DB productiva.

26. Flujos end-to-end

26.1 QUERY

```
User -> Owl panel
  -> ScreenContext + authenticated request
  -> Assistant Service
  -> ContextBuilder
  -> EffectiveModelSchema
  -> Codex asks odoo.aggregate/search
  -> ToolExecutor -> AddonDelegationGateway
  -> Odoo ORM as user
  -> Evidence(record/aggregate)
  -> Codex AnswerEnvelope
  -> UI + evidence chips
```

26.2 DIAGNOSE

```
User: "¿Por qué no se valida esta factura?"
  -> current account.move
  -> runtime fields/modules
  -> LogProvider search around timestamp/error
  -> traceback fingerprint
  -> source symbol for custom frame
  -> excerpt + config metadata
  -> Codex separates checked error from hypotheses
  -> answer with record/log/source evidence
```

26.3 HOW_TO

```
User: "¿Cómo configuro plazo de pago 30 días?"
  -> current model/context
  -> find menu paths + relevant model schema
  -> search client knowledge
  -> optional source/custom evidence
  -> explain exact steps for this installation
  -> if asked, offer safe ACTION preview to perform change
```

26.4 ACTION

```
User: "Cambia el comercial de estos presupuestos a Laura"
-> resolve selected_ids
-> resolve Laura under user permissions
-> preview_update
-> UI shows exact change
-> user approves
-> commit with payload-bound Approval
-> Odoo ORM write as user
-> re-read + audit
-> final result
```

27. Estrategia de testing

27.1 Pirámide mínima

Nivel	Qué prueba	Ejemplos
Unit	parsers/policy/contracts/redactor	manifest AST, domain validation, payload hash
Contract	ports/adapters	OdooGateway, LogProvider, ReasoningEngine fake
Integration	PostgreSQL + Odoo 18 real	fields_get, record rules, delegated reads/writes
E2E	UI -> service -> Odoo -> Codex fake/real	vertical slice, approval flow
Compatibility	mismo contract suite por major	Odoo 18 baseline; Odoo 19 después

27.2 Fixture vertical slice

- Odoo 18 CE con sale + project.
- Addon fixture que sobrescribe sale.order.action_confirm y crea/actualiza project.task bajo condición visible.
- Usuarios fixture: admin técnico, usuario sales limitado y usuario sin acceso a campos concretos.
- Logs controlados con traceback representativo.
- Preguntas golden: causa correcta, falta de permiso, staleness tras cambiar hash.

27.3 Tests de seguridad obligatorios

- restricted field no aparece en tool schema ni respuesta.
- record rule oculta registros al agente igual que al usuario.
- DelegationContext expirado/replay rechazado.
- write sin approval o con payload modificado rechazado.
- business action no allowlisted rechazada.
- Codex isolation self-test no puede leer source/log paths por shell.
- prompt injection fixture en comentario/log/documento no cambia tool policy.

28. Observabilidad

28.1 Trace lógico

```
trace_id
turn.started
screen_context.captured
policy.resolved
context.prepared
engine.started
tool.requested
tool.completed
```

```
evidence.added
engine.completed
answer.created
approval.previewed / approved / committed (if any)
turn.completed
```

28.2 Qué almacenar

Sí	No por defecto
timestamps, ids, workflow, tool names, latency, sizes, status	passwords, API keys, delegation secrets
evidence ids/pointers y fragmentos usados	prompts completos indefinidamente
error class + sanitized traceback del assistant	logs completos del host
token/cost usage cuando engine lo exponga	raw config con secretos

28.3 UI de Diagnostics

- Últimos turns fallidos con trace_id.
- Tool que falló, adapter/provider y error sanitizado.
- Readiness/capability report.
- Scan status/fingerprint.
- Botones: test logs, test source, test Codex, reanalizar.

29. Roadmap y milestones

Milestone	Objetivo	Resultado observable
M0 - Repo/contratos	monorepo, docs, contratos mínimos, CI local	tests unit verdes; arquitectura congelada
M1 - Runtime/install	FastAPI, Postgres, migrations, bootstrap, health	service instalable y detectado desde Odoo
M2 - UI/context/delegation	panel, ScreenContext, signed delegation, read_record	preguntar desde pedido y leerlo como usuario real
M3 - Source + logs	scanner mínimo + File/Journal LogProvider	buscar action_confirm y traceback desde Diagnostics
M4 - Codex agent slice	Codex login, dynamic tools, ContextPack, AnswerEnvelope	"¿por qué confirmar crea tarea?" resuelto E2E con evidencia
M5 - QUERY + HOW_TO + RAG	search/aggregate, menus/views, knowledge FTS	consultas y guías adaptadas a la instalación
M6 - ACTION segura	preview/create/update + approval + 1 business action	cambio simple desde chat y verificado
M7 - Product hardening	installer UX, staleness, tracing, limits, security tests	piloto utilizable por técnico sin consola diaria
M8 - Odoo 19	fixtures/compat/UI adapter; JSON-2 sólo si se necesita	misma contract suite y workflows sin diff

29.1 Dependencias

- M2 depende de M1.
- M3 puede avanzar en paralelo parcialmente con M2 una vez exista storage.
- M4 requiere M2 + M3 y produce el primer vertical slice que valida producto/arquitectura.
- M5 y M6 sólo empiezan después de que M4 sea estable; evitan expandir una base no validada.
- M8 no debe bloquear el valor de Odoo 18.

30. Criterios de aceptación

Milestone	Done significa
M0	No hay imports vendor/version en application; contracts exportan JSON Schema; pytest baseline.
M1	Instalador crea/actualiza runtime y assistant DB; health visible desde Odoo; rollback documentado.
M2	Usuario limitado sólo puede leer lo que Odoo le permite; ScreenContext cambia al navegar; no hay sudo.
M3	Scan encuentra método fixture y líneas; re-scan invalida hash; logs encuentran traceback en ventana acotada.
M4	Vertical slice identifica módulo, método, condición y evidencia exacta sin shell libre ni repo completo en prompt.
M5	QUERY usa agregación server-side; HOW_TO valida menú/schema local; docs RAG devuelve fuentes correctas.
M6	Payload alterado tras preview no se ejecuta; approval one-time; commit se relee y audita.
M7	Un técnico instala/actualiza con un único bootstrap y opera después desde Odoo; Diagnostics explica fallos.
M8	Tests contract sobre Odoo 19 verdes; application/services sin checks de mayor nuevos.

31. Decisiones arquitectónicas

ID	Decisión	Alternativas descartadas / motivo
ADR-001	Addon + Assistant Service local obligatorio	Addon-only mezcla LLM/scanner/logs y dificulta aislamiento.
ADR-002	Assistant DB PostgreSQL separada	SQLite añade segundo motor; DB/schema Odoo acopla lifecycle y seguridad.
ADR-003	Sin SQL directo a Odoo	ORM conserva ACL/rules/semántica.
ADR-004	ReasoningEngine port + Codex inicial	Evita dependencia estructural del proveedor.
ADR-005	Runtime schemas	Instalación real depende de módulos/custom/grupos, no sólo mayor.
ADR-006	Capabilities de instancia separadas de policy de request	Evita confundir "instancia puede escribir" con "usuario puede escribir".
ADR-007	Tools estrechas + agent loop	Autonomía útil sin shell/execute_method libre.
ADR-008	Lexical/structural retrieval primero	Menos infraestructura y mejor precisión en símbolos Odoo.
ADR-009	Source + logs obligatorios para FULLY_READY	Diferencial de diagnóstico y requisito explícito del producto.
ADR-010	Instalador privilegiado único, no root en Odoo	Automatiza sin convertir el proceso ERP en administrador del host.
ADR-011	Browser sólo habla con Odoo	Simplifica auth/CORS y mantiene UX Odoo-native.
ADR-012	Approval ligado al payload	Evita confirmaciones desacopladas o TOCTOU

ID	Decisión	Alternativas descartadas / motivo
		de operación.
ADR-013	Repo propio, reutilización selectiva	Ningún repo existente cubre a la vez UX, identidad, source/logs y licencia/alcance deseado.

32. Decisiones pospuestas

Tema	Cuándo decidir
Embeddings/pgvector	Sólo si evals de docs muestran recall insuficiente con FTS.
MCP como integración Codex	Si dynamicTools experimentales generan inestabilidad o si otro engine lo requiere.
Multi-instance / central service	Cuando exista necesidad real de partner fleet/tenant isolation.
Odoo.sh/Docker adapters completos	Después del piloto self-hosted y al priorizar deployments objetivo.
Policy engine sofisticado	Cuando allowlists/toggles no expresen reglas reales.
Delete/unlink	Cuando haya casos de negocio y UX de riesgo definida.
Generación de módulos	Fuera del MVP; requiere threat model y workflow distinto.
Modelo/provider comercial definitivo	Después de validar valor y coste del agent loop con Codex.
Streaming SSE directo	Si polling de eventos no ofrece UX suficiente.

33. Riesgos conocidos

Riesgo	Impacto	Mitigación / señal
AST/XML no explica causalidad dinámica	diagnóstico incompleto	cruzar runtime/logs; medir coverage en custom reales
Log correlation débil	falsas causalidades	etiquetar inferencia; usar ventanas/traceback/request ids
Codex dynamicTools experimental	cambio de API	adapter aislado; fallback futuro MCP
Codex filesystem isolation inconsistente	bypass de tools	Linux permission profile + self-test fail-closed; endurecer antes de clientes
fields_get/schema cache por usuario	fuga metadata	EffectiveModelSchema por security context + TTL corto
custom methods internos usan sudo	acción puede ampliar permisos	business actions curadas y tests; no confiar sólo en ACL CRUD
instalación en hostings diversos	setup tosco	perfil self-hosted primero; bootstrap detecta y reporta excepciones
RAG responde genérico	pérdida de confianza	priorizar evidencia local y marcar general knowledge
licencias de donors	bloqueo comercial	provenance de código + revisión por archivo antes de copiar
coste/latencia agent loop	UX/coste	budgets, pre-context determinista, aggregate

Riesgo	Impacto	Mitigación / señal
		server-side, evals

34. Estrategia y prompts para Codex

34.1 Regla de ejecución

Principio. Codex implementa milestones pequeños sobre un repo inspeccionado. No recibe "implementa todo". Cada task packet fija contratos que no puede romper, áreas permitidas, tests y acceptance criteria.

34.2 Plantilla de task packet

PROMPT CODEX - MILESTONE <ID>

Contexto

- Lee docs/ARCHITECTURE.md y ADRs relevantes.
- Inspecciona el repo real antes de modificar.

Objetivo

- <resultado observable único>

Contratos que NO puedes romper

- <files/contracts>

Debes reutilizar

- <componentes existentes/donors ya incorporados>

Debes implementar

- <scope concreto>

Fuera de scope

- <lista explícita>

Restricciones

- no sudo / no direct Odoo SQL / no version checks in application
- preserve current-user semantics

Tests obligatorios

- <commands>

Acceptance criteria

- <checks concretos>

Antes de editar

1. Resume brevemente el estado real del repo.
2. Señala cualquier conflicto con el Source of Truth.

Después

1. Ejecuta tests.
2. Informa archivos cambiados, decisiones y riesgos pendientes.
3. No declares done si falta una verificación.

34.3 PROMPT CODEX - M0

Objetivo: crear el esqueleto del monorepo y contratos mínimos sin implementar features.

Implementa:

- service/ pyproject y package layout
- contracts: ScreenContext, RecordRef, Evidence, ContextPack, ToolSpec, AnswerEnvelope
- ports: ReasoningEngine, OdooGateway, LogProvider
- ARCHITECTURE.md con invariantes de este documento

- ADR-001..ADR-004 resumidos
- pytest + lint/type baseline

No implementes:

- scanner real
- Codex
- Odoo addon funcional
- registries/plugins sofisticados

Acceptance:

- imports respetan boundaries
- JSON Schemas exportables
- tests verdes

34.4 PROMPT CODEX - M1

Objetivo: runtime instalable y observable.

Implementa:

- FastAPI /health y /v1/admin/status
- PostgreSQL SQLAlchemy + Alembic
- tablas mínimas instance_profile, capability_snapshot, trace_event
- installer bootstrap idempotente
- systemd unit, service config y shared secret file
- addon placeholder detectable

Acceptance:

- fresh Ubuntu/Linux host de test -> un bootstrap -> service healthy
- segunda ejecución idempotente
- assistant role no puede CONNECT a Odoo DB

34.5 PROMPT CODEX - M2

Objetivo: primera lectura contextual Odoo end-to-end.

Implementa:

- Owl systray + panel mínimo
- ScreenContext service
- signed DelegationContext
- internal addon tool endpoint
- OdooGateway read/current_record + fields_get
- EffectiveModelSchema sin cache persistente

Tests:

- usuario restricted
- field group restriction
- record rule
- expired/replayed delegation

Acceptance:

- desde sale.order abierto, /turn debug devuelve RecordSnapshot real con permisos correctos.

34.6 PROMPT CODEX - M3

Objetivo: source + logs como evidencia determinista.

Implementa:

- scan de installed addon roots: manifest literal, Python AST, XML, CSV
- source_file/source_symbol/xml_record persistence
- source.find_symbol/read_excerpt
- FileLogProvider + JournalLogProvider contract
- logs.search/read_traceback
- redaction de secretos básica

Fixture:

- custom sale addon que crea project.task en action_confirm

Acceptance:

- find_symbol devuelve líneas correctas
- scan incremental por hash
- traceback fixture recuperable por ventana/terms

34.7 PROMPT CODEX - M4 vertical slice

Objetivo: responder E2E "¿Por qué confirmar este pedido crea una tarea?".

Implementa:

- CodexAppServerEngine por stdio
- device-code login endpoints/Settings
- ContextBuilder
- dynamicTools read-only para Odoo/source/logs
- outputSchema AnswerEnvelope
- agent loop + budgets + trace events
- conversation/turn/evidence persistence mínima

Restricciones:

- Codex no recibe el repo completo
- no shell tool como mecanismo de investigación
- sólo evidencia con refs válidas

Acceptance:

- identifica módulo, action_confirm, condición y project.task
- respuesta cita source exacto y record actual
- cambiar source + rescane evita respuesta stale

34.8 PROMPT CODEX - M5

Objetivo: convertir el slice en asistente funcional read-only.

Implementa:

- QUERY: DomainSpec, search_records, aggregate
- HOW_TO: find_menu_paths, inspect_view
- knowledge sources + chunks + PostgreSQL FTS
- routing/policy simple por workflow
- conversation summaries

Acceptance:

- deuda/facturas se calcula server-side
- HOW_TO usa menú/schema real
- knowledge fixture con vocabulario distinto se recupera con FTS baseline o documenta miss medible

34.9 PROMPT CODEX - M6

Objetivo: ACTION segura mínima.

Antes de implementar:

- audita ERPipe safe-write code MIT y documenta qué se adapta/copia.

Implementa:

- WriteRequest/Preview/Approval
- preview_create/preview_update
- payload canonical hash + expiry + one-time consume
- write.commit vía OdooGateway
- UI preview/confirm
- una business action explícita (handler, no generic method)

Acceptance:

- mutated payload rejected
- no approval rejected
- restricted user denied by Odoo
- successful write re-read and audited

34.10 PROMPT CODEX - M7/M8

M7: endurecer producto
 - installer UX, Diagnostics, readiness, caps, security/e2e, docs operativas.

M8: Odoo 19
 - crear fixtures reales/sanitizados
 - ejecutar contract suite
 - adaptar UI/version profile sólo donde falle
 - implementar JSON-2 adapter únicamente si una ruta externa real lo necesita
 - demostrar que application/ no cambia.

35. Fuentes y referencias

35.1 Documentos de entrada

- D1 - Odoo_AI_Assistant_Viabilidad_Disenio_2026(1).pdf. Investigación técnica y diseño inicial, 20-08-2026.
- D2 - Odoo_AI_Arquitectura_Dinamica_Schemas_Configs_2026(1).pdf. Arquitectura dinámica por schemas/capabilities, 20-08-2026.

35.2 Odoo oficial

O18-SEC - Odoo 18 - Security in Odoo:

<https://www.odoo.com/documentation/18.0/developer/reference/backend/security.html>

O18-ORM - Odoo 18 - ORM API / fields_get:

<https://www.odoo.com/documentation/18.0/developer/reference/backend/orm.html>

O18-API - Odoo 18 - External API / ir.model.fields:

https://www.odoo.com/documentation/18.0/developer/reference/external_api.html

O18-WEB - Odoo 18 - Frontend framework / registries:

https://www.odoo.com/documentation/18.0/developer/reference/frontend/framework_overview.html

O18-REG - Odoo 18 - Registries (systray/main_components/services):

<https://www.odoo.com/documentation/18.0/developer/reference/frontend/registries.html>

O19-AI - Odoo 19 - AI agents: <https://www.odoo.com/documentation/19.0/applications/productivity/ai/agents.html>

O19-CTX - Odoo 19 - AI default prompts / record context:

https://www.odoo.com/documentation/19.0/applications/productivity/ai/default_prompts.html

O19-ACT - Odoo 19 - AI server actions / manager-worker:

<https://www.odoo.com/documentation/19.0/applications/productivity/ai/server-actions.html>

O19-JSON2 - Odoo 19 - External JSON-2 API:

https://www.odoo.com/documentation/19.0/developer/reference/external_api.html

O19-RPC - Odoo 19 - External RPC deprecation:

https://www.odoo.com/documentation/19.0/developer/reference/external_rpc_api.html

35.3 Codex / OpenAI

CX-APP - OpenAI Codex app-server README:

<https://github.com/openai/codex/blob/main/codex-rs/app-server/README.md>

CX-PERM - Codex core permissions/sandbox references: <https://github.com/openai/codex/tree/main/codex-rs>

35.4 Repositorios analizados

GH-ERPIPE - erpipe-org/mcp-odoo (MIT): <https://github.com/erpipe-org/mcp-odoo>

GH-VAUXOO - Vauxoo/mcp.odoo (MIT): <https://github.com/Vauxoo/mcp.odoo>

GH-APEX - apexive/odoo-llm (LGPL-3): <https://github.com/apexive/odoo-llm>

GH-OCA - OCA/ai (repo AGPL-3; revisar licencia de cada módulo): <https://github.com/OCA/ai>

GH-CLAW - nicolasramos/odooclaw (MIT): <https://github.com/nicolasramos/odooclaw>

Nota de actualidad. Las APIs, licencias y repositorios pueden cambiar. Antes de copiar código o cerrar soporte

comercial para una nueva versión, verificar de nuevo las fuentes originales. Las decisiones de este documento se basan en el estado observado el 20-08-2026.

35.5 Checklist de inicio inmediato

- Crear repo/branch y copiar este documento a docs/ como referencia.
- Ejecutar M0 con Codex; no implementar features antes de validar boundaries.
- Auditar código MIT de ERPipe para scanner/write/query helpers antes de M3/M6.
- Preparar entorno Odoo 18 fixture desde el principio.
- Hacer que M4 vertical slice sea la primera gran puerta de inversión: si no aporta precisión/valor, corregir antes de crecer.